

PROGRAMA CURRICULAR DE INGENIERÍA AGRÍCOLA
PROGRAMACIÓN PARA ANÁLISIS DE ESTRUCTURAS

CONCEPTOS Y FUNDAMENTOS SOBRE PROGRAMACIÓN DE COMPUTADORES

Presentado por:
Luis Octavio González Salcedo
Profesor Titular



¿Qué entendemos por Programación?

La Programación es el proceso de diseñar, organizar y expresar una secuencia lógica de instrucciones que permite a un computador ejecutar tareas y contribuir a la solución de un problema determinado

De un problema a un programa

¿qué necesitamos resolver?

PROBLEMA

¿cómo podemos resolverlo?

PROCEDIMIENTO

¿Qué pasos debe ejecutar el computador?

INSTRUCCIONES

Las instrucciones se expresan mediante un lenguaje de computador

PROGRAMA

El computador procesa los datos y realiza las operaciones indicadas

EJECUCIÓN

La solución obtenida debe ser analizada e interpretada

RESULTADO

Programar = construir una solución lógica que el computador pueda ejecutar

¿Por qué programar en Ingeniería?

La ingeniería requiere resolver problemas que pueden involucrar numerosos datos, operaciones repetitivas, procedimientos iterativos y modelos matemáticos. La programación permite convertir estos procedimientos en soluciones que pueden ser ejecutadas sistemáticamente por un computador

AUTOMATIZAR

REPETIR

ACCELERAR

SISTEMATIZAR

RESOLVER PROBLEMAS COMPLEJOS

La computadora ejecuta rápidamente las instrucciones; la ingeniería está en formular correctamente el problema y el procedimiento para resolverlo

Del problema a la solución computacional

FASE 1 – RESOLUCIÓN DEL PROBLEMA

1.- ANÁLISIS

2.- SOLUCIÓN GENERAL

3.- ALGORITMO

4.- VERIFICACIÓN

FASE 2 – IMPLEMENTACIÓN COMPUTACIONAL

5.- CODIFICACIÓN E IMPLEMENTACIÓN

6- EJECUCIÓN Y COMPROBACIÓN

Primero se diseña la solución, después se programa

¿Qué es un algoritmo?

Algoritmo: Definido como la secuencia ordenada de pasos, sin ambigüedades, que conducen a la solución de un problema dado y expresado en lenguaje natural.

1.- **Preciso:** indica el orden de realización de cada uno de los pasos.

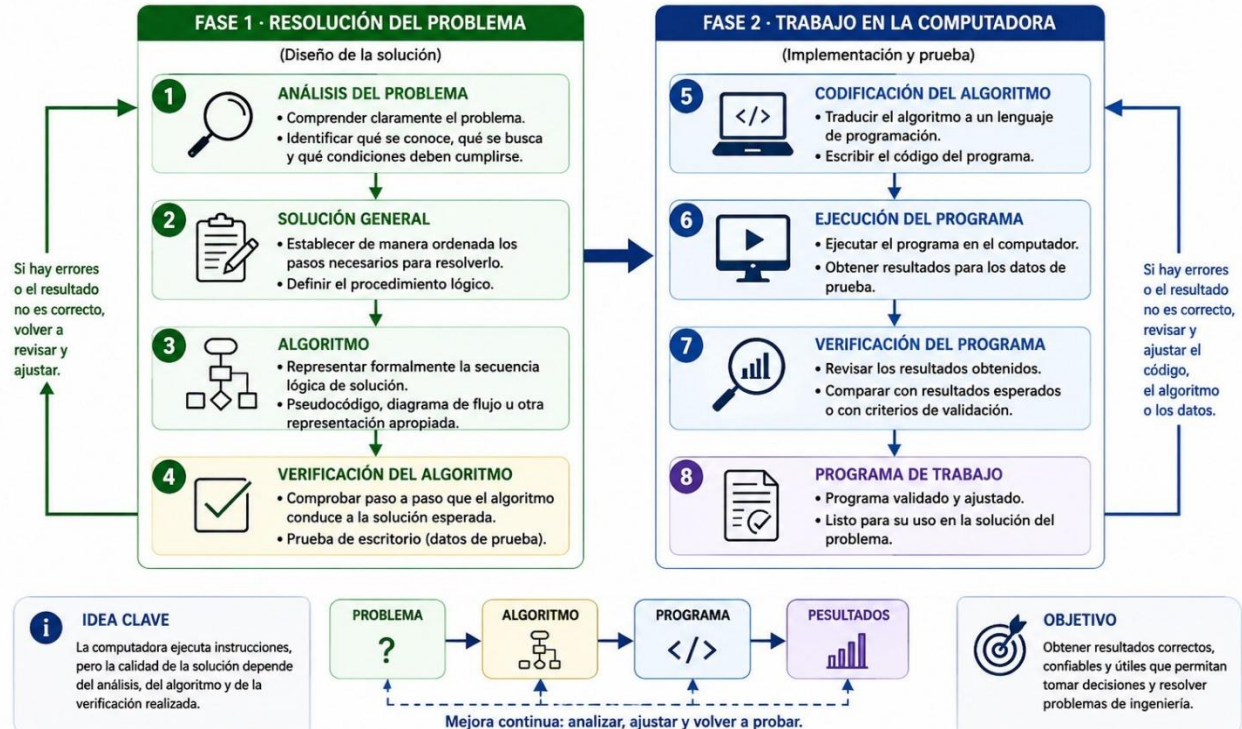
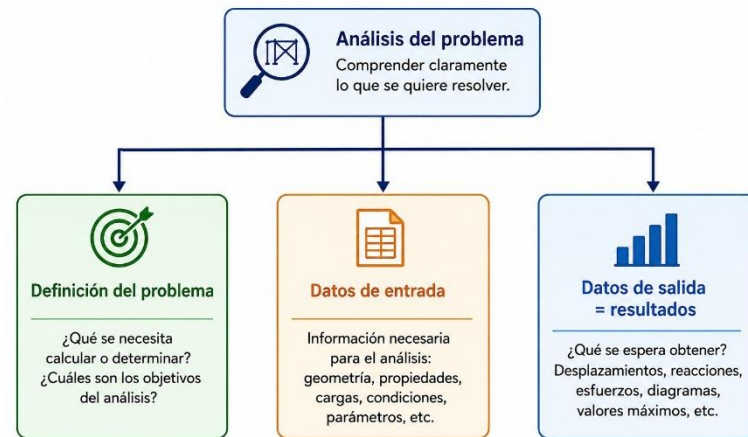
2.- **Definido:** deben conducir a los mismos resultados para iguales datos de entrada.

3.- **Finito:** el algoritmo termina en algún momento.

a.- **Entrada:** información dada al algoritmo.

b.- **Procesos:** operaciones o cálculos.

c.- **Salida:** respuestas dadas por el algoritmo.



Herramientas de Programación:

Diagrama de flujo

Es la representación gráfica de las operaciones ejecutadas sobre los datos a través de todas las partes de un sistema de procesamiento de información (diagrama de flujo del sistema), así como, de la secuencia de pasos necesarios para describir un procedimiento particular (diagrama de flujo de detalles).

Utiliza símbolos normalizados, con los pasos del algoritmo escritos en el símbolo adecuado y los símbolos unidos por flechas, denominadas líneas de flujo, que indican el orden en que los pasos deben ser ejecutados.

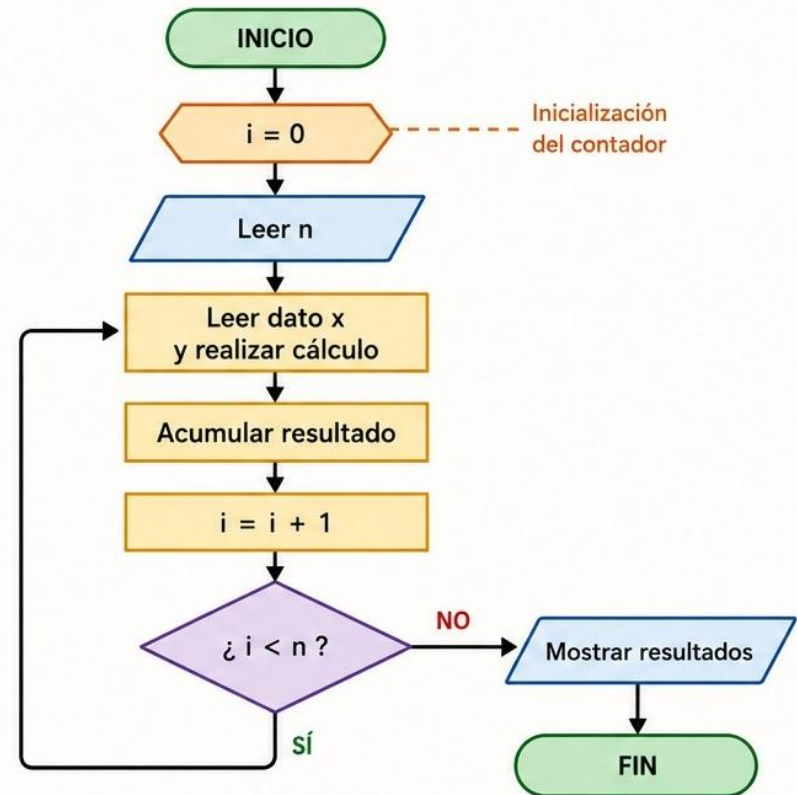
Herramientas de Programación

Símbolos del diagrama de flujo

SÍMBOLOS BÁSICOS

SÍMBOLO	NOMBRE	FUNCIÓN
	INICIO / FIN	Indica el inicio o la terminación del proceso.
	ENTRADA / SALIDA	Representa la entrada de datos o la salida de información.
	PROCESO	Indica una operación, cálculo o instrucción cualquiera.
	DECISIÓN	Evalúa una condición y genera dos o más alternativas.
	LÍNEA DE FLUJO	Indica la dirección del flujo del proceso.
	CONECTOR	Permite enlazar partes del diagrama dentro de la misma página.
	CONECTOR DE PÁGINA	Permite enlazar partes del diagrama en otras páginas.
	PREPARACIÓN / INICIALIZACIÓN	Establece valores iniciales o condiciones previas para el proceso. Ejemplo: contador $i = 0$

EJEMPLO DE DIAGRAMA DE FLUJO (CON CONTADOR)



IDEA CLAVE: Los diagramas de flujo permiten visualizar la lógica de un algoritmo de manera clara y comprensible antes de escribir el código en un lenguaje de programación.



Herramientas de Programación

Pseudocódigo

Es una forma de representar un algoritmo mediante instrucciones escritas en lenguaje sencillo y estructurado, sin depender de la sintaxis de un lenguaje de programación específico.

¿Para qué sirve?

Organizar la solución paso a paso.

Comprender la lógica antes de programarlo.

Verificar y detectar errores en la secuencia solución.

Traducir y facilitar posteriormente en un lenguaje de programación.

SECUENCIA
DECISIÓN
REPETICIÓN

SECUENCIA

Leer datos

Calcular resultado

Mostrar resultado

DECISIÓN

SI condición ENTONCES

Realizar acción

SI_NO

Realizar otra acción

FIN_SI

REPETICIÓN

MIENTRAS Condición HACER

Realizar acciones

FIN_MIENTRAS

PROBLEMA → ALGORITMO → PSEUDOCÓDIGO → CÓDIGO

Herramientas de Programación

Tres formas de expresar una solución

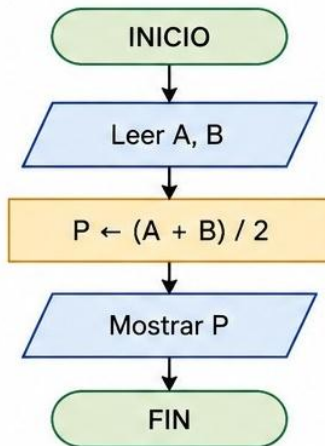
La lógica del algoritmo es la misma; lo que cambia es la forma de representarla



PROBLEMA: Leer dos números, calcular su promedio y mostrar el resultado.

1 DIAGRAMA DE FLUJO

Representación gráfica



Permite visualizar gráficamente la secuencia del algoritmo.

2 PSEUDOCÓDIGO

Representación textual estructurada

```
INICIO
  LEER A, B
  P ← (A + B) / 2
  MOSTRAR P
FIN
```



Describe la lógica del algoritmo sin depender de un lenguaje de programación específico.

3 CÓDIGO (Python)

Representación ejecutable

```
1 A = float(input("A: "))
2 B = float(input("B: "))
3
4 P = (A + B) / 2
5
6 print(P)
```



Expresa el algoritmo utilizando las reglas de un lenguaje de programación.



IDEA DE SOLUCIÓN



DIAGRAMA DE FLUJO



PSEUDOCÓDIGO



CÓDIGO



EJECUCIÓN



MISMA LÓGICA, distintas formas de representación para comprender, diseñar y ejecutar soluciones.

Herramientas de Programación

Lenguajes de Programación

1.- Conjunto de reglas o normas, símbolos y palabras especiales utilizadas para construir un programa, y con él darle solución a un problema.

2.- Es el encargado de que la computadora realice paso a paso las tareas que el programador ha diseñado en el algoritmo.

3.- Es el intermediario entre la máquina y el usuario para que éste pueda resolver problemas a través de las computadoras para la realización de dicho trabajo.

Los lenguajes de programación se pueden clasificar por:

1.- Clasificación histórica → Generaciones.

2.- Clasificación en niveles → Nivel de Abstracción & Arquitectura de Computación.

3.- Clasificación por paradigmas → Distinción de Modelos de Cómputo & Estilos de Estructuración y Organización.

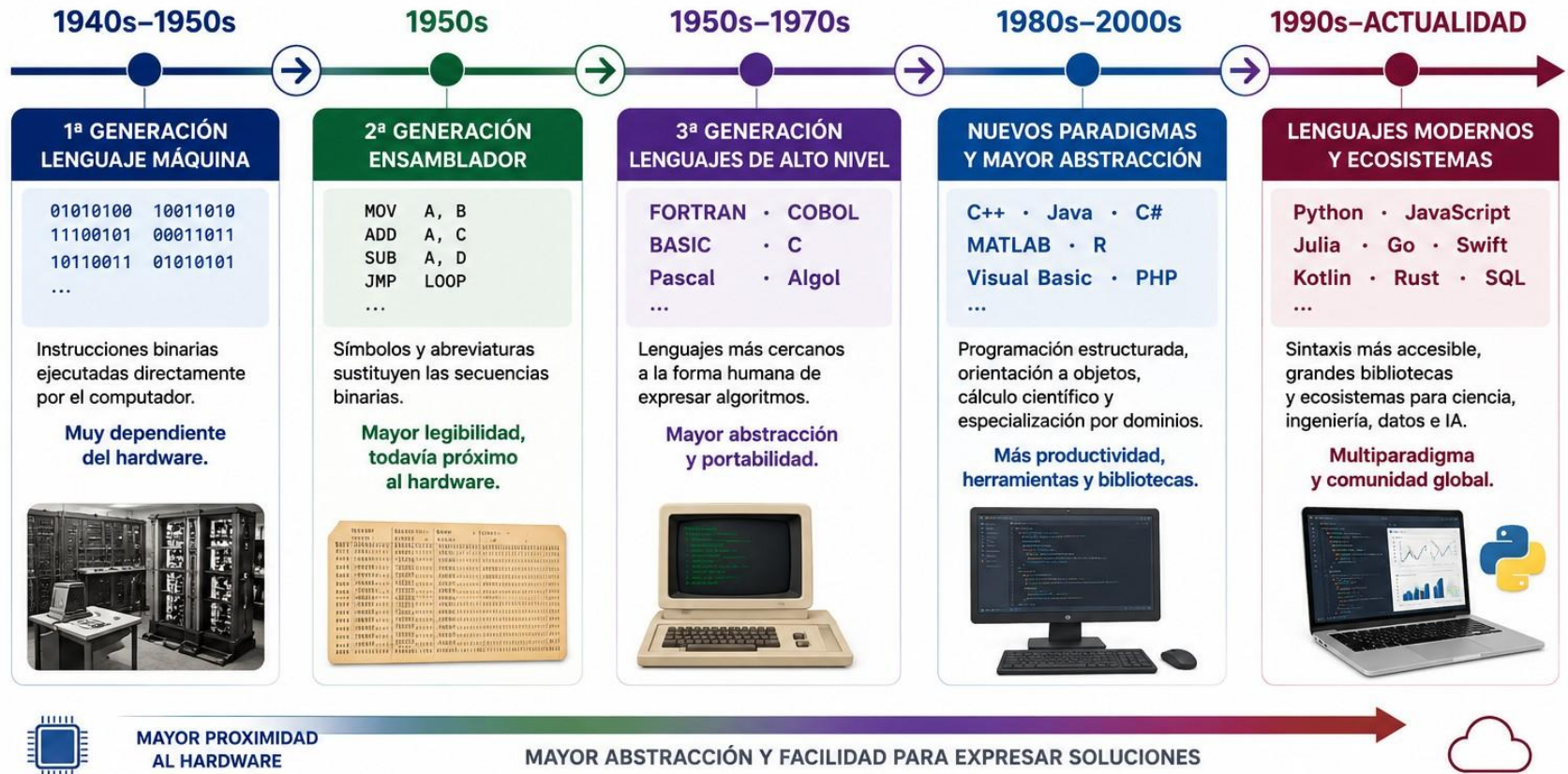
4.- Clasificación por propósitos → Propósito General & Propósito Específico.

Herramientas de Programación

Evolución de los lenguajes de programación

EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN

Del lenguaje máquina a los lenguajes modernos: mayor abstracción, más poder, más soluciones.



IMPORTANTE: Las generaciones no reemplazan a las anteriores. Lenguaje máquina y ensamblador siguen existiendo y son fundamentales.



Lo que cambia es el nivel de abstracción: menos detalles del hardware, **más foco en resolver problemas.**

Paradigmas de la programación

Un paradigma de programación es una forma de concebir, organizar y estructurar la solución de un problema mediante un programa.

Paradigma	Idea central	Pregunta intuitiva
Imperativo / procedural	Organiza la solución como una secuencia de instrucciones y procedimientos que modifican el estado del programa.	¿Qué pasos debo ejecutar?
Orientado a objetos	Organiza el programa mediante objetos que integran datos y comportamientos.	¿Qué objetos intervienen y qué pueden hacer?
Funcional	Construye soluciones mediante funciones y transformación de datos, favoreciendo expresiones y funciones reutilizables.	¿Qué transformaciones aplico a los datos?
Lógico	Expresa hechos, reglas y relaciones; la solución se obtiene mediante inferencia.	¿Qué condiciones y relaciones deben cumplirse?
Multiparadigma	Permite combinar diferentes paradigmas según las características del problema.	¿Qué enfoque resulta más conveniente?

Python es multiparadigma. Python permite trabajar, entre otros enfoques, con programación: procedural + orientada a objetos + funcional

Elementos de Programación

Variables y Arreglos: títulos asignados a espacios de memoria para almacenar datos específicos.

Variables ... Numéricas, Cadenas de Caracteres, Alfanuméricas, binarias.

Arreglos ... Vectores, Matrices.

Tipo de dato	Descripción
Char	Contienen un único carácter, es decir, una letra, un signo.
Int	Contienen un número entero.
Float	Contienen un número real.
String	Contienen cadenas de texto, es decir un vector con varias variables de tipo Char.
Boolean	Solo pueden contener un cero o un uno. En algunos lenguajes, “False” y “True”, respectivamente.

Condicionales: son sentencias que indican que, para que cierta parte del programa se ejecute, deben cumplirse ciertas premisas.

IF ... THEN ...

ELSE ...

Ó ELSE IF ...

Bucles: son sentencias que ejecutan constantemente el código, mientras se cumple una condición.

FOR I NEXT I

WHILE ... THEN ...

Funciones: dividir un problema complejo en tareas

Una función es un conjunto organizado de instrucciones destinado a realizar una tarea específica.

¿QUÉ ES UNA FUNCIÓN?



¿POR QUÉ UTILIZAR FUNCIONES?



DIVIDIR PROBLEMAS COMPLEJOS

Permite descomponer en tareas más pequeñas.



ORGANIZAR LA SOLUCIÓN

Estructura clara y modular del algoritmo.



REUTILIZAR PROCEDIMIENTOS

La misma función puede usarse muchas veces.



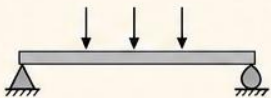
FACILITAR VERIFICACIÓN Y MANTENIMIENTO

Más fácil comprobar, depurar y modificar.

EJEMPLO EN ANÁLISIS ESTRUCTURAL: PROBLEMA COMPLEJO → CONJUNTO DE FUNCIONES

PROBLEMA COMPLEJO

Analizar una viga con cargas y apoyos.

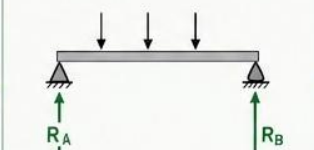


Calcular reacciones, cortante, momento y deflexión.



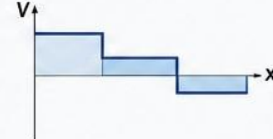
FUNCIÓN 1

Calcular reacciones



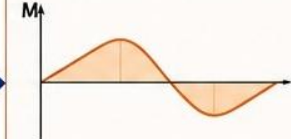
FUNCIÓN 2

Calcular cortante



FUNCIÓN 3

Calcular momento



FUNCIÓN 4

Calcular deflexión



IDEA CLAVE: Un problema complejo se resuelve como un conjunto de funciones (tareas) que reciben datos específica y devuelven resultados. realizan una operación específica.



DESCOMPONER



ORGANIZAR



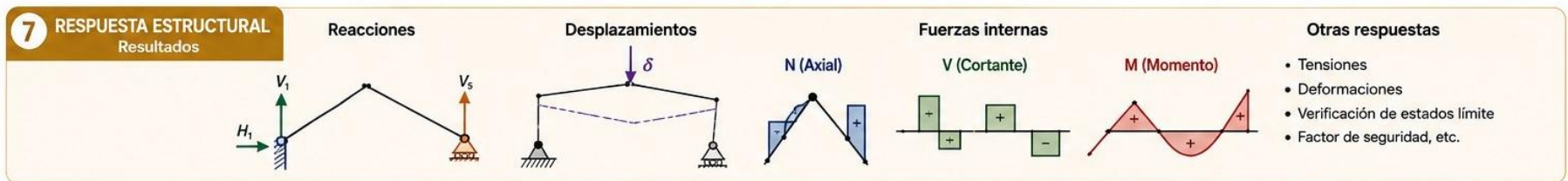
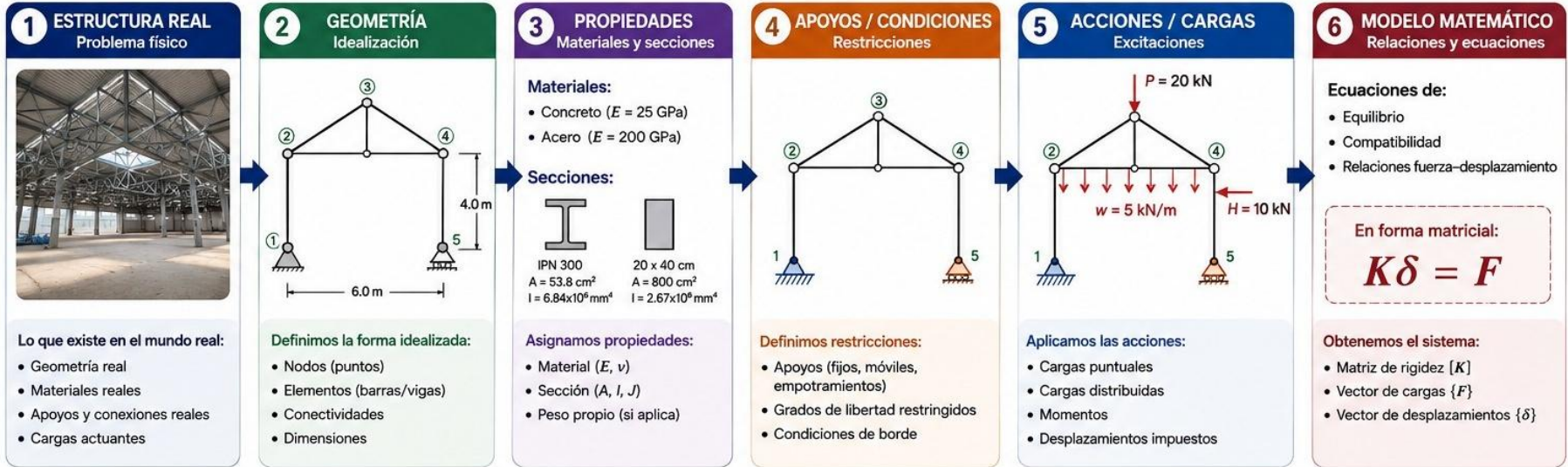
REUTILIZAR



VERIFICAR

¿Cómo se representa un problema estructural?

Transformamos la estructura real en un modelo matemático que el computador pueda procesar.



EN RESUMEN: DE LO FÍSICO A LO COMPUTACIONAL



IDEA CLAVE

Un buen programa estructural comienza con una adecuada **REPRESENTACIÓN** del problema. Sin datos correctos, no hay resultados confiables.



Representar un problema estructural es definir **qué información** necesita el modelo, **cómo se organiza** y **cómo se relaciona** para obtener la respuesta.

¿Qué información manejará un programa estructural?

Los programas estructurales trabajan con diferentes tipos de datos para representar y resolver el modelo.

1 ESCALARES

Un solo valor

Cantidad representada por un único número.

Ejemplos en estructuras

- Módulo de elasticidad $E = 200 \text{ GPa}$
- Área de una sección $A = 0.015 \text{ m}^2$
- Inercia $I = 8.50 \times 10^{-6} \text{ m}^4$
- Longitud de un elemento $L = 6.00 \text{ m}$
- Carga puntual $P = 20 \text{ kN}$

Representación

$$E = 200\text{e}9$$

$$A = 0.015$$

$$I = 8.50\text{e}-6$$

$$L = 6.00$$

$$P = 20000$$

Un valor numérico



Parámetros, propiedades, dimensiones, cargas puntuales, etc.

2 VECTORES

Conjunto ordenado de valores

Lista de valores relacionados entre sí.

Ejemplos en estructuras

- Vector de cargas nodales F
- Vector de desplazamientos δ
- Coordenadas de un nodo (x, y, z)
- Reacciones en apoyos (H, V, M)

Representación

Cargas nodales $F \text{ (kN)}$	Desplazamientos $\delta \text{ (mm)}$	Coordenadas $(x, y, z) \text{ (m)}$
$\begin{bmatrix} 0 \\ -15 \\ 0 \\ 10 \\ 0 \\ -5 \end{bmatrix}$	$\begin{bmatrix} 0.00 \\ 0.12 \\ -0.05 \\ 0.00 \\ 0.08 \\ -0.02 \end{bmatrix}$	$\begin{bmatrix} (0.00, 0.00, 0.00) \\ (6.00, 0.00, 0.00) \\ (6.00, 4.00, 0.00) \\ (0.00, 4.00, 0.00) \\ (3.00, 6.00, 0.00) \end{bmatrix}$



Almacenan magnitudes distribuidas en nodos o elementos.

3 MATRICES

Valores organizados en filas y columnas

Arreglo bidimensional de números que describe relaciones.

Ejemplos en estructuras

- Matriz de rigidez global K
- Matriz de masa M
- Matriz de transformación T
- Matriz de flexibilidad F

Representación

Matriz de rigidez global K

$$K = \begin{bmatrix} k_{11} & k_{12} & \dots & k_{1n} \\ k_{21} & k_{22} & \dots & k_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ k_{n1} & k_{n2} & \dots & k_{nn} \end{bmatrix}$$

Simétrica para estructuras lineales
($k_{ij} = k_{ji}$)



Relacionan fuerzas y desplazamientos.
Base del modelo: $F = K \delta$

4 TABLAS / CONJUNTOS DE DATOS

Información organizada en registros

Colecciones de datos relacionados por filas y columnas.

Ejemplos en estructuras

- Tabla de nodos (ID, coordenadas, apoyos)
- Tabla de elementos (conectividad, sección)
- Propiedades de materiales y secciones
- Cargas distribuidas por elemento

Representación

Nodos					Elementos			
ID	x (m)	y (m)	z (m)	Apoyo	ID	Nodo i	Nodo j	Sección
1	0.00	0.00	0.00	Fijo	1	1	3	IPN 300
2	6.00	0.00	0.00	Rodillo	2	3	2	IPN 300
3	6.00	4.00	0.00	—	3	1	4	IPN 300
...	...	...	...	...	4	4	3	IPN 300

Materiales				Cargas			
ID	Material	E (GPa)	ν	ID	Tipo	Valor	Dónde
1	Concreto	25	0.20	1	q (kN/m)	5.00	Elem. 2
2	Acero	200	0.30	2	P (kN)	20.00	Nodo 3



Permiten almacenar y gestionar gran cantidad de información de manera ordenada.



EN SÍNTESIS

Un programa estructural manipula información de diferentes formas para describir el modelo y obtener su respuesta.

Escalares

Parámetros y propiedades

$E, A, I, L, P, \dots$

Vectores

Magnitudes en nodos / elementos

$F, \delta, (x, y, z), \dots$

Matrices

Relaciones entre fuerzas y desplazamientos

$K, M, T, \dots$

Tablas / Datos

Información organizada del modelo

Nodos, elementos, materiales, cargas, ...

Modelo completo de la estructura



PUENTE CON LA ECUACIÓN CLAVE

Todos estos datos se integran para resolver:

$$F = K \delta$$

Fuerzas (vector) Matriz de rigidez (matriz) Desplazamientos (vector)



IDEA CLAVE: La naturaleza del problema estructural determina qué información necesitamos y cómo debe organizarse para que el computador pueda procesarla.



Problema físico



Representación computacional



Información organizada (escalares, vectores, matrices, tablas)



Operaciones del modelo (equilibrio, compatibilidad, relaciones)



Respuesta estructural (reacciones, desplazamientos, fuerzas internas, etc.)

Entrada de datos

¿Cómo recibe un programa la información que necesita?

Los datos de entrada proporcionan al programa la información necesaria para ejecutar el procedimiento y obtener resultados.

1 DATOS INCORPORADOS EN EL PROGRAMA



Valores definidos directamente para un problema particular.

Ejemplo:

$E = 200 \text{ GPa}$
 $L = 5.00 \text{ m}$
 $P = 20 \text{ kN}$
 $A = 0.015 \text{ m}^2$
 $\nu = 0.30$

Definidos en el código fuente.

2 ENTRADA DEL USUARIO



Datos suministrados durante la ejecución del programa.

Ejemplo (interacción):

- > Ingrese longitud de la viga (m): 6.00
- > Ingrese carga puntual P (kN): 20
- > Ingrese módulo E (GPa): 200
- > Ingrese tipo de apoyo (1=Emp, 2=Art): 2

El programa solicita los datos al usuario.

3 ARCHIVOS DE DATOS



Información almacenada en archivos de texto estructurados.

Formatos comunes:

CSV • TXT • JSON • ...

Ejemplo (CSV – nodos y cargas):

Nodo	x (m)	y (m)	Fx (kN)	Fy (kN)
1	0.00	0.00	0.0	0.0
2	4.00	0.00	0.0	-20.0
3	4.00	3.00	10.0	0.0
4	0.00	3.00	0.0	0.0
...	...	...	...	...

Archivos portables e intercambiables.

4 HOJAS DE CÁLCULO (Excel)



Datos organizados en filas y columnas.

Ejemplo (Excel):

ID	Elemento	A (m ²)	E (GPa)	ρ (kg/m ³)
1	1-2	0.015	200	7850
2	2-3	0.020	200	7850
3	3-4	0.015	200	7850
4	4-1	0.020	200	7850
...	...	...	...	...

Muy útil para preparar, editar y administrar datos.

5 BASES DE DATOS / FUENTES EXTERNAS



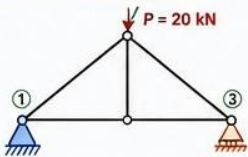
Grandes conjuntos de información almacenados y consultados sistemáticamente.

Ejemplo:

- Base de datos de materiales
- Registros de ensayos
- Monitoreo estructural
- Inventario de elementos
- Normas y catálogos

Permiten consultas eficientes y actualizadas.

EJEMPLO: DATOS PARA UN MODELO ESTRUCTURAL



- Geometría (nodos, elementos)
- Propiedades (E , A , sección)
- Apoyos y restricciones
- Cargas y condiciones

Toda esta información es la **ENTRADA** del programa.

PROGRAMA



lee

organiza

verifica

procesa

Convierte los datos en información útil para el análisis.

RESULTADOS



Cálculos (numéricos)



Tablas



Diagramas (V , M , N , δ)



Archivos de salida

Información procesada y lista para usar.



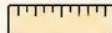
ANTES DE PROCESARLOS, LOS DATOS DEBEN SER VERIFICADOS

Tipo de dato

123 abc 1.2

numérico, texto, booleano, etc.

Unidades



consistentes y correctas

Dimensiones



concordancia geométrica

Consistencia



relaciones físicas y lógicas

Valores faltantes o atípicos



detección y tratamiento



Datos confiables → Resultados confiables

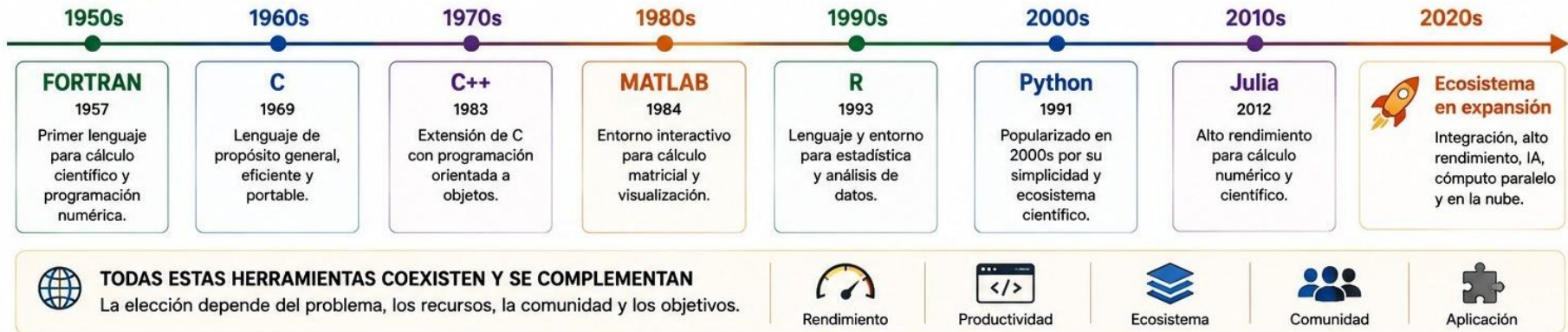


Un buen programa estructural no solo calcula bien, también **lee, organiza y verifica** correctamente los datos de entrada.

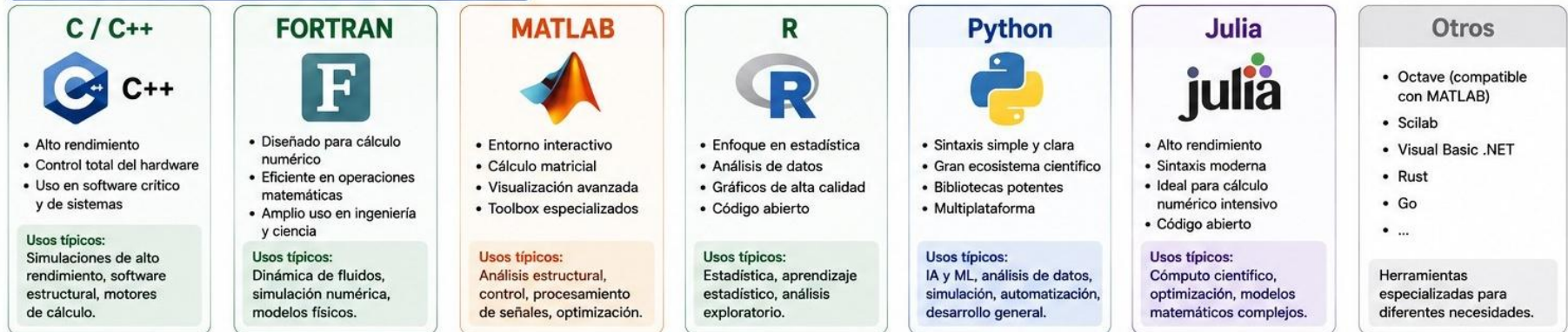
Lenguajes y herramientas de cálculo científico

Herramientas que permiten formular modelos, procesar datos y resolver problemas complejos.

PANORAMA HISTÓRICO Y CONTEMPORÁNEO



LENGUAJES Y HERRAMIENTAS MÁS UTILIZADOS HOY



¿CÓMO ELEGIR LA HERRAMIENTA ADECUADA?

- ✓ Naturaleza del problema (tamaño, complejidad, tipo de análisis).
- ✓ Requerimientos de rendimiento y memoria.
- ✓ Disponibilidad de bibliotecas y herramientas.
- ✓ Comunidad y soporte.
- ✓ Integración con otras herramientas y flujos de trabajo.

EN INGENIERÍA ESTRUCTURAL



- Cualquiera de estas herramientas puede utilizarse para:
- Análisis estructural
 - Métodos numéricos (MEF, diferencias finitas, etc.)
 - Optimización de secciones y materiales
 - Procesamiento de datos experimentales
 - Automatización de tareas de diseño y verificación



IDEA CLAVE

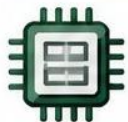
No existe un "mejor" lenguaje o herramienta, sino la más adecuada para cada problema y cada momento.

Un buen ingeniero domina los conceptos y elige sabiamente la herramienta.

¿Dónde se ejecutan los cálculos? CPU/GPU/TPU

El programa se ejecuta en hardware que realiza las operaciones numéricas.

PARA NUESTRO CURSO



CPU

(Central Processing Unit)
Procesador central

Diseñada para ejecutar una amplia variedad de tareas de forma secuencial y con gran flexibilidad.

- ✓ Pocos núcleos muy potentes (4 - 16+ núcleos típicos)
- ✓ Excelente para tareas secuenciales y lógica compleja
- ✓ Menor paralelismo masivo
- ✓ Uso eficiente de memoria



SUFICIENTE PARA NUESTRO CURSO

Análisis estructural, métodos numéricos, optimización, procesamiento de datos, etc.



GPU

(Graphics Processing Unit)
Procesador gráfico

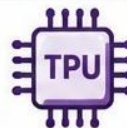
Diseñada para realizar muchísimas operaciones en paralelo de manera muy eficiente.

- ✓ Miles de núcleos más simples
- ✓ Ideal para cómputo paralelo masivo
- ✓ Muy alta capacidad de procesamiento
- ✓ Mayor consumo de energía
- ✓ Requiere programación o librerías especializadas (CUDA, OpenCL, etc.)



ACELERADOR ESPECIALIZADO

Útil para IA, simulaciones masivas, gráficos 3D, análisis de grandes datasets.



TPU

(Tensor Processing Unit)
Procesador tensorial

Diseñada específicamente para acelerar cargas de trabajo de aprendizaje automático (deep learning).

- ✓ Arquitectura específica para tensores
- ✓ Máxima eficiencia en modelos de IA
- ✓ Alta paralelización con bajo consumo (en la nube)
- ✓ Uso principalmente en plataformas específicas (Google Cloud TPU)



ACELERADOR ESPECIALIZADO

Optimizado para redes neuronales y aprendizaje profundo.

COMPARACIÓN GENERAL



Flexibilidad



Paralelismo



Facilidad de uso (programación)



Consumo de energía



Disponibilidad



Costo

CPU

★★★★★

★★

★★★★★

★★★★★

★★★★★

★

GPU

★★★★

★★★★★

★★★★

★★

★★★★

★★★★

TPU

★

★★★★★

★★

★★★★★

★★★

★★



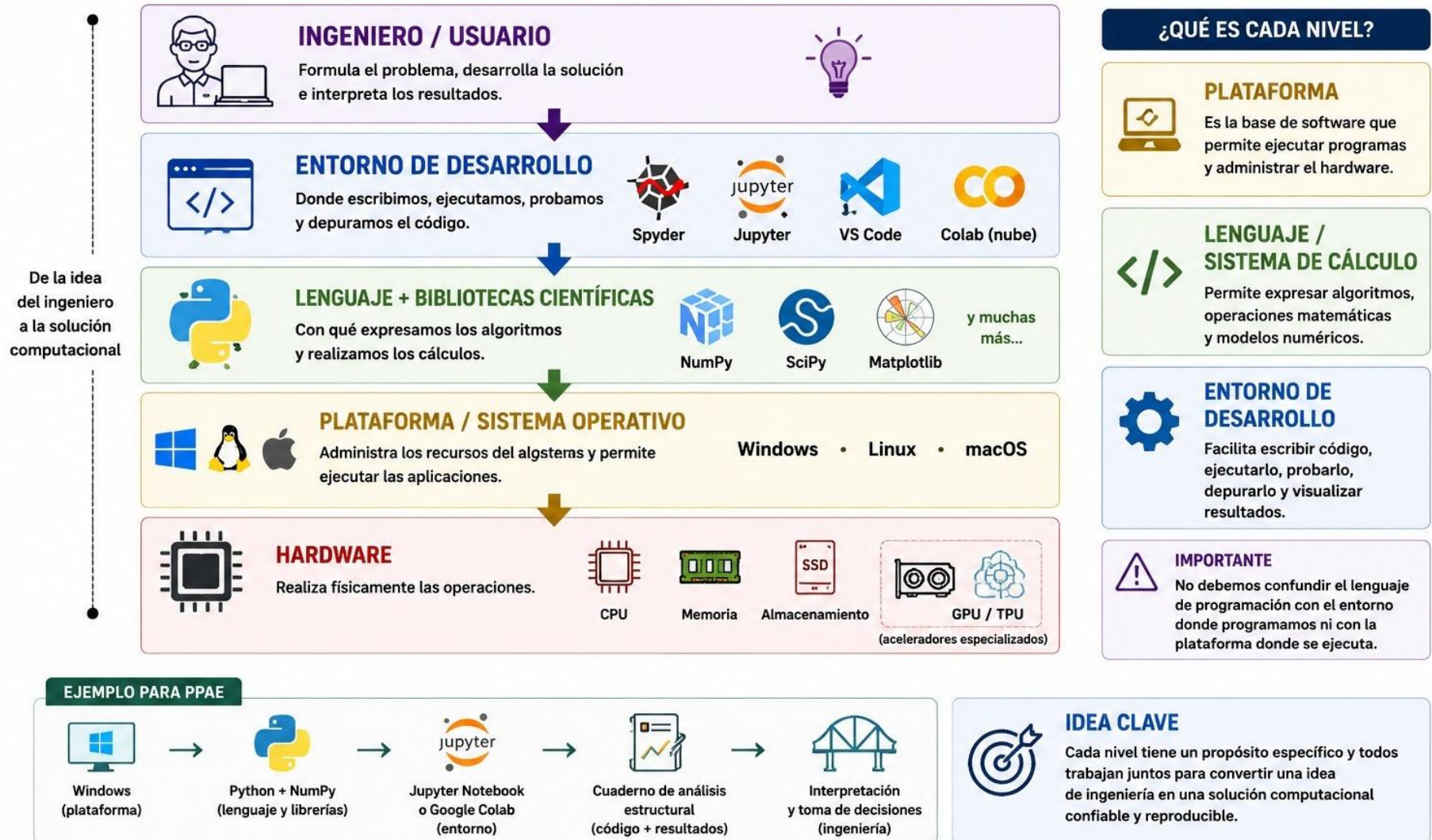
IDEA CLAVE: Para la mayoría de problemas de ingeniería estructural y métodos numéricos de este curso, una CPU moderna es más que suficiente. GPU y TPU son aceleradores especializados para tareas de alto paralelismo o inteligencia artificial a gran escala.

Plataformas, sistemas de cálculo y entornos de programación

Para desarrollar y ejecutar una solución computacional intervienen diferentes niveles de software, cada uno con una función distinta.

Nivel	¿Qué es?	Ejemplos	Función
Plataforma / sistema operativo	Base sobre la cual funcionan las aplicaciones y se administran los recursos del computador.	Windows · Linux · macOS	Proporciona el entorno general de ejecución
Lenguaje / sistema de cálculo científico	Herramienta con la que expresamos y ejecutamos operaciones, algoritmos y modelos numéricos.	Python · MATLAB · R · Julia	Permite formular y resolver el problema computacional
Entorno de desarrollo	Interfaz y conjunto de herramientas que facilitan escribir, ejecutar, probar y depurar programas.	Spyder · VS Code · Jupyter Notebook	Facilita el trabajo del programador

Plataformas, sistemas de cálculo y entornos de programación



Jupyter Project y los cuadernos computacionales

JUPYTER PROJECT

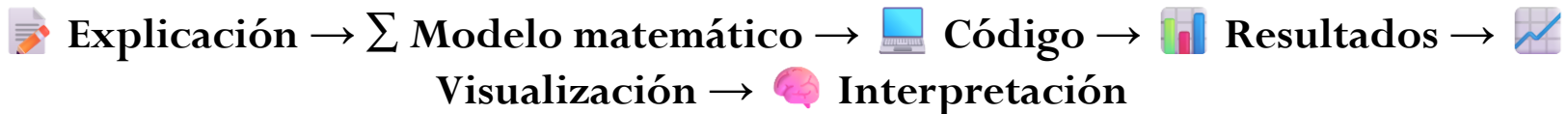
Proyecto de código abierto orientado a la computación interactiva.

Permite trabajar con cuadernos computacionales, integrando en un mismo documento:

Texto · ecuaciones · código · datos ·
resultados · gráficos

CUADERNO COMPUTACIONAL

Documento interactivo y ejecutable que combina la explicación de un problema con los procedimientos utilizados para resolverlo y sus resultados.



DOCUMENTO TRADICIONAL

Texto + ecuaciones + figuras

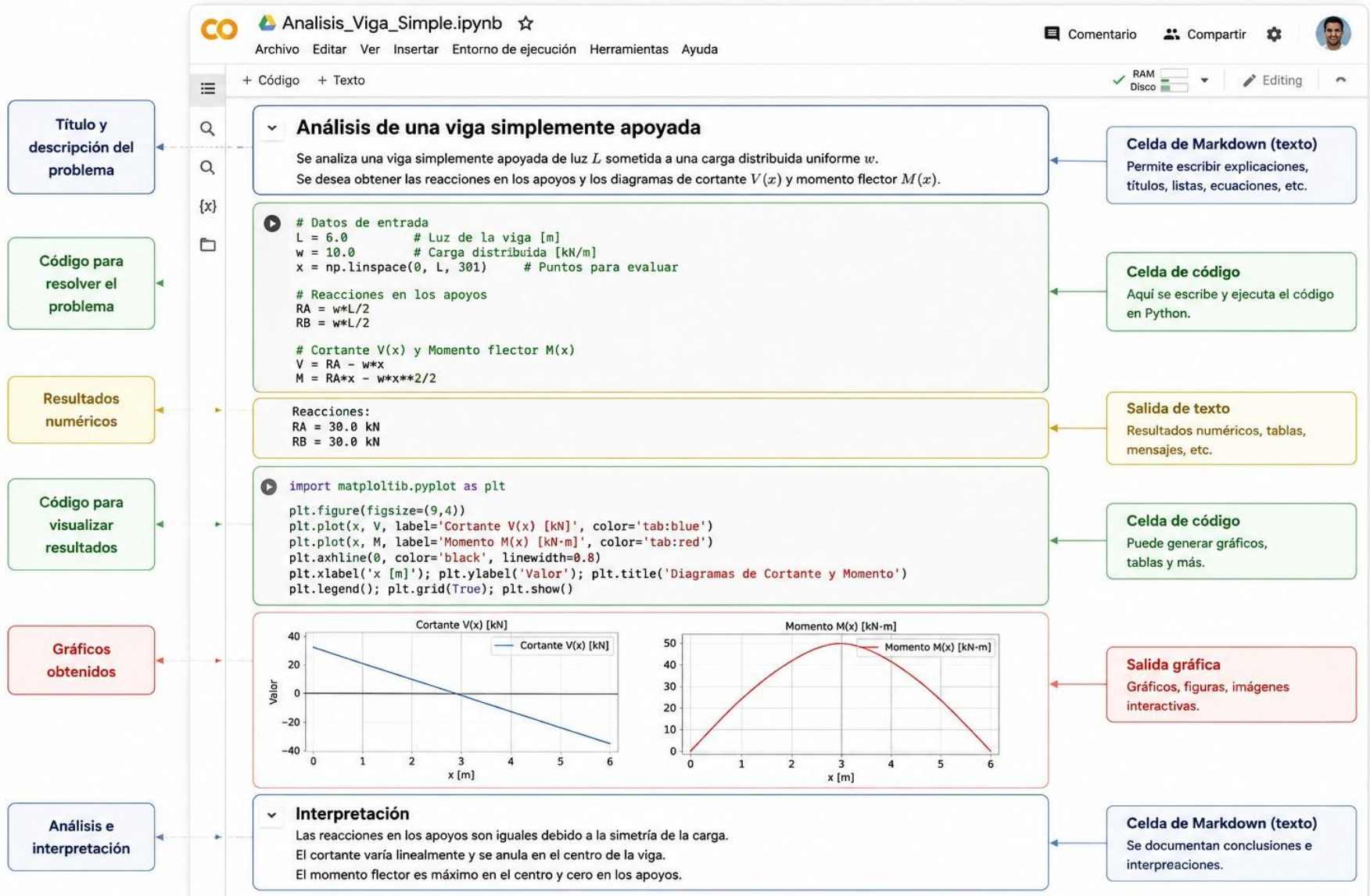
CUADERNO COMPUTACIONAL

Texto + ecuaciones + código ejecutable +
resultados + gráficos + interpretación

El cuaderno permite documentar, ejecutar, modificar, verificar y reproducir una solución computacional.

Anatomía de un Cuaderno Computacional

Ejemplo de un cuaderno (Jupyter/Colab)

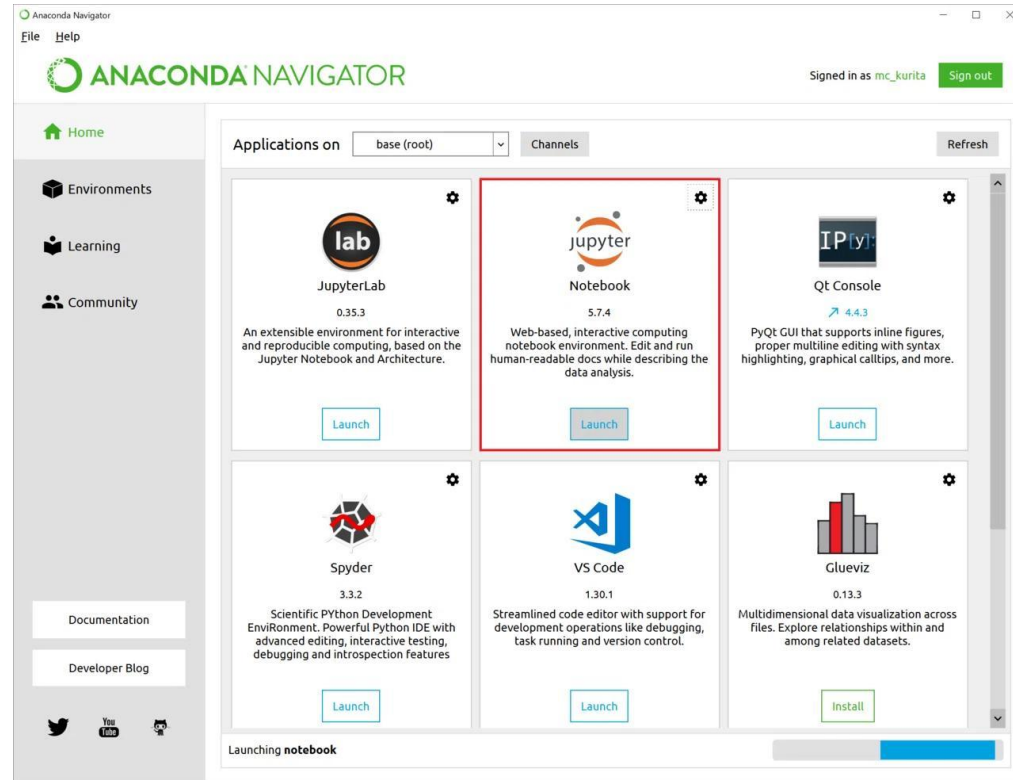


Jupyter: el entorno de los cuadernos

Jupyter permite crear y ejecutar cuadernos computacionales combinando documentación, código y resultados

Sus principales productos desarrollados y soportados, de computación interactiva, corresponden a Jupyter Notebook, Jupyter Kernels, Jupyter Hub, y Jupyter Lab.

Notebook es un entorno informático interactivo basado en la web para crear documentos (cuadernos) que pueden contener código, texto (usando lenguaje Markdown), matemáticas, gráficos y textos enriquecidos.



Google Colaboraty: Cuadernos en la Nube

Entorno de cuadernos computacionales que permite escribir y ejecutar Python desde un navegador, sin necesidad de instalaciones locales.

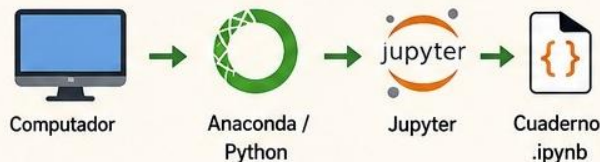
¿CÓMO FUNCIONA?



¿QUÉ NOS OFRECE?

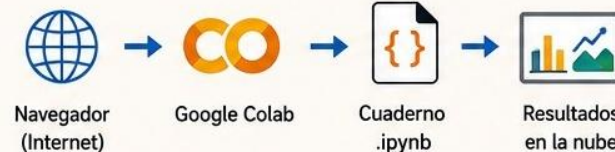
- SIN INSTALACIÓN LOCAL**
No necesitas instalar Python ni librerías en tu computador.
- ENTORNO PREPARADO**
Python y muchas bibliotecas científicas ya están disponibles.
- CUADERNOS JUPYTER**
Trabaja con el formato .ipynb, el mismo que en Jupyter.
- EJECUCIÓN EN LA NUBE**
Los cálculos se ejecutan en recursos computacionales remotos.
- INTEGRACIÓN**
Puede trabajar con archivos en Google Drive y cuadernos alojados en GitHub.

TRABAJO LOCAL (COMPUTADOR)



VS.

TRABAJO EN LA NUBE (COLAB)



EN PPAE

Google Colab será nuestro laboratorio computacional principal para desarrollar y ejecutar los cuadernos del curso.



MISMO CONCEPTO DE CUADERNO → DIFERENTE LUGAR DE EJECUCIÓN

Escribes, ejecutas y analizas en un cuaderno Jupyter, pero ahora en la nube.



En resumen:

Colab = Jupyter + Python + computación en la nube.

Google Drive: almacenamiento y organización

Espacio en la nube para almacenar, organizar y acceder a los archivos utilizados durante el trabajo computacional.



ALMACENAR

Guarda tus archivos en la nube.



ORGANIZAR

Crea carpetas y organiza tus contenidos.



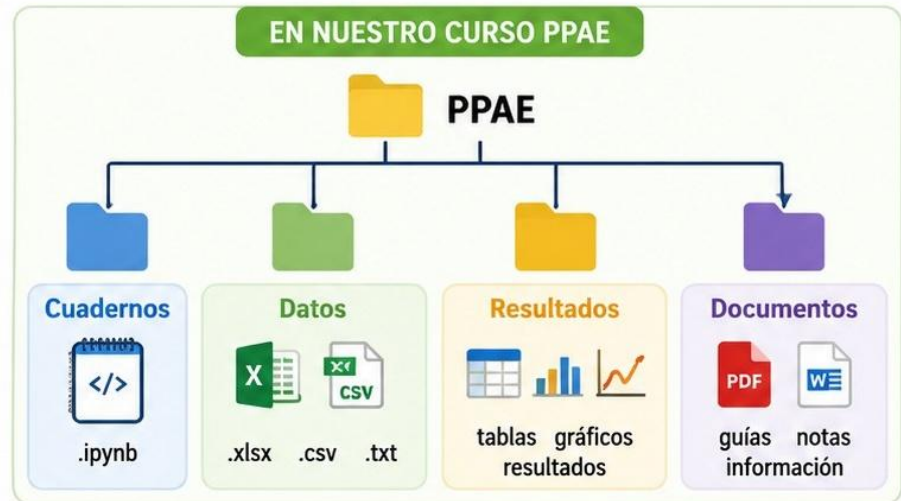
ACCEDER

Accede desde diferentes equipos y lugares.



COMPARTIR

Comparte archivos y carpetas de forma segura.



GOOGLE COLAB ↔ GOOGLE DRIVE

Abrir cuaderno

Leer datos



Guardar cuaderno

Guardar resultados



IDEA CLAVE

DRIVE = ALMACENAMIENTO Y ORGANIZACIÓN

No es nuestro sistema de **control de versiones** del código.



¿Entonces cómo mantenemos historial, versiones y compartimos proyectos de programación?



GIT + GITHUB

Control de versiones, trazabilidad y colaboración



Drive será nuestro espacio de almacenamiento y organización; no nuestro sistema de control de versiones.



Git & Github: control de versiones, trazabilidad y colaboraciones

Permiten registrar la evolución del código y trabajar en equipo de manera organizada y segura.



GIT

Sistema de control de versiones

Gestiona el historial de cambios en tu proyecto.

PERMITE:



Registrar cambios



Conservar historial



Comparar versiones



Recuperar estados anteriores



IDEA CLAVE

Git **registra** la evolución.

GitHub **conserva** y **comparte** esa evolución.



GITHUB

Plataforma en la nube para alojar repositorios Git

Facilita la colaboración y la distribución de proyectos.

PERMITE:



Almacenar repositorios



Compartir y colaborar

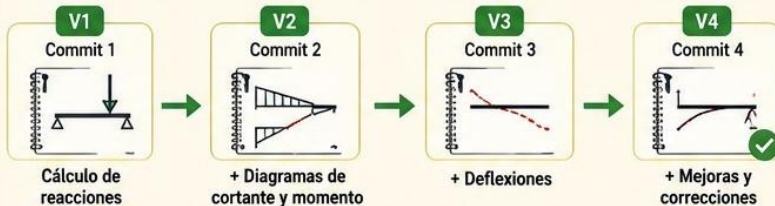


Documentar proyectos



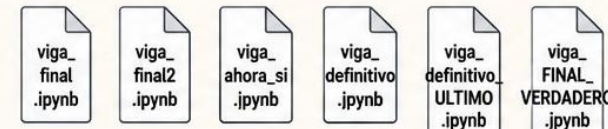
Publicar y dar visibilidad

✓ USANDO GIT: HISTORIAL ORDENADO Y TRAZABLE



VS.

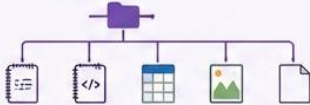
✗ SIN CONTROL DE VERSIONES: CAOS DE ARCHIVOS



¿Cuál es la última?

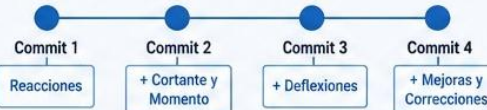
REPOSITORIO

Es el proyecto completo y su historial. Contiene todos los archivos y carpetas relacionados.



VERSIÓN / COMMIT

Es un registro identificado de cambios realizados en el tiempo.



HISTORIAL

Permite conocer qué cambió, quién lo hizo y cuándo, y recuperar versiones anteriores.



Almacena y organiza tus archivos en la nube (tu espacio personal).



Registra y controla la evolución de tu proyecto (historias de cambios).



Aloja y comparte tus repositorios, colabora y da visibilidad.



TRABAJO ORDENADO, TRAZABLE Y COLABORATIVO

Código con historial • Recuperación de versiones
Colaboración en equipo • Proyectos profesionales

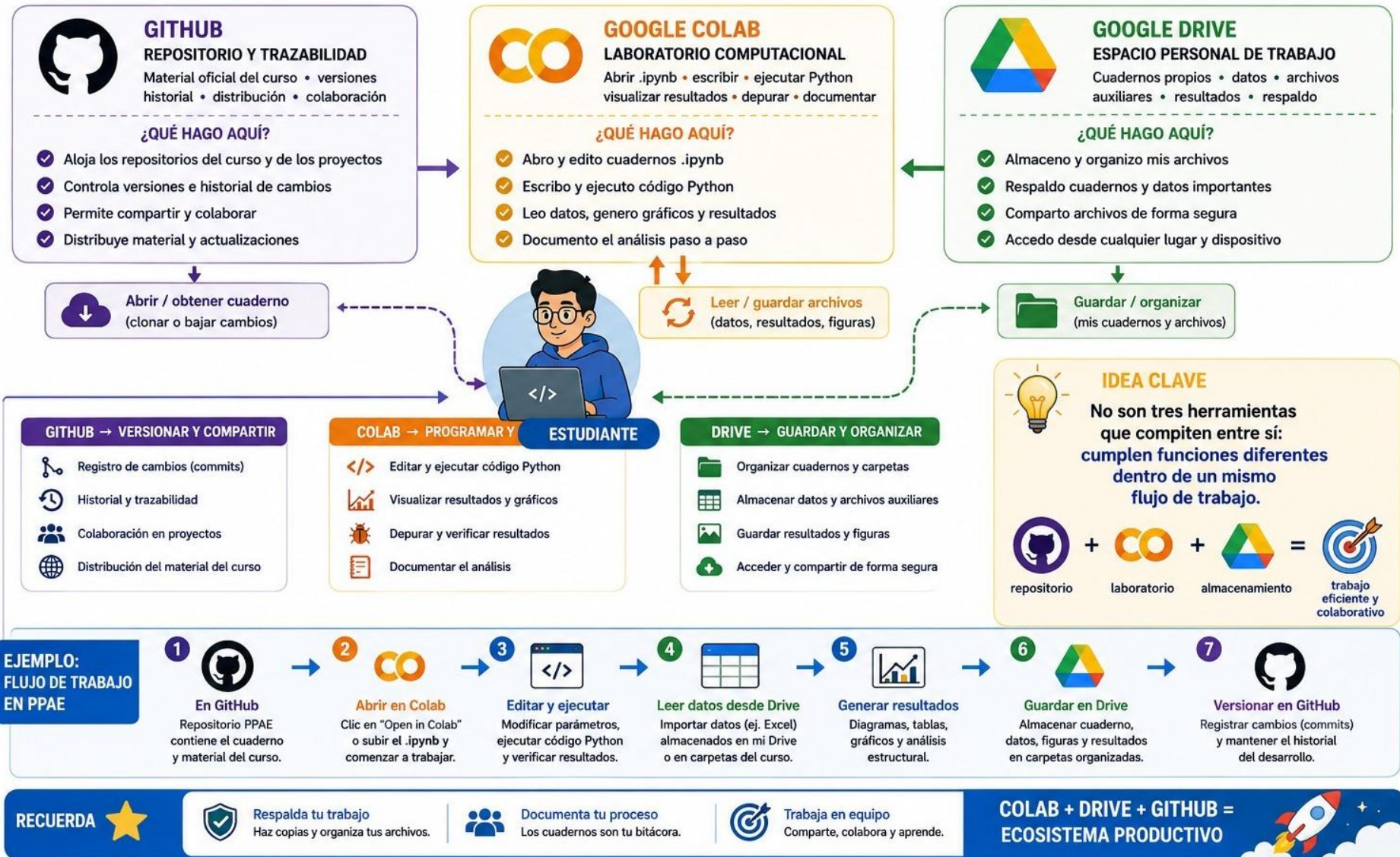


GIT Y GITHUB son la base para un desarrollo profesional: control, calidad y colaboración.



Colab+Drive+Github: ecosistema de trabajo

Tres herramientas, tres funciones complementarias, un solo flujo de trabajo.



La IA como asistente de programación

Una aliada poderosa para programar, aprender y resolver problemas de ingeniería... **¡con criterio y responsabilidad!**



IA GENERATIVA PUEDE ASISTIR...

- ✓ **PROPONER** Algoritmos y alternativas de solución.
- ✓ **GENERAR** Borradores de código a partir de una especificación.
- ✓ **EXPLICAR** Código, procedimientos, conceptos y errores.
- ✓ **DEPURAR** Ayudar a localizar errores y proponer correcciones.
- ✓ **MODIFICAR** Adaptar código a nuevas condiciones o requerimientos.
- ✓ **DOCUMENTAR** Comentarios, reportes y documentación técnica.
- ✓ **APRENDER** Acompañar el proceso de estudio y práctica.

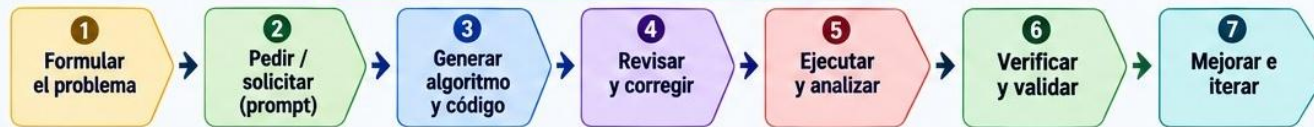
Un trabajo en equipo...



INGENIERO CONSERVA LA RESPONSABILIDAD...

- ✓ **FORMULAR** Definir correctamente el problema de ingeniería.
- ✓ **MODELAR** Establecer hipótesis, variables y relaciones.
- ✓ **ESPECIFICAR** Proporcionar datos, condiciones, restricciones y objetivos.
- ✓ **VERIFICAR** Comprobar código, unidades, ecuaciones y resultados.
- ✓ **VALIDAR** Determinar si la solución representa adecuadamente el fenómeno físico.
- ✓ **INTERPRETAR Y DECIDIR** Dar significado ingenieril a los resultados.

FLUJO DE TRABAJO CON IA GENERATIVA



UNA BUENA SOLICITUD TÉCNICA (PROMPT) PROPORCIONA ...



Ejemplo de prompt para PPAE:

"Analiza una viga simplemente apoyada de 6 m sometida a una carga distribuida de 10 kN/m. Propón primero el algoritmo, después el código Python y verifica las reacciones mediante equilibrio estático. Genera los diagramas de cortante y momento."

Pedir: "Hazme un programa de una viga"
⇒ suele producir resultados pobres.

IDEA CLAVE

"La IA generativa no reemplaza al ingeniero, lo potencia."

- ✓ Acelera el aprendizaje.
- ✓ Mejora la productividad.
- ✓ Permite explorar más alternativas.
- ✓ Ayuda a entender y depurar el código.



RECUERDE:

- ✓ Código que ejecuta ≠ código correcto
- ✓ Código correcto ≠ solución de ingeniería válida
- ✓ La solución debe verificarse e interpretarse con criterio técnico del ingeniero.



LA IA PUEDE GENERAR CÓDIGO. EL INGENIERO DEBE VERIFICAR LA SOLUCIÓN.

La IA generativa se incorpora como asistente dentro del proceso de solución de problemas en PPAE.



La IA como asistente de programación

31 ¿CÓMO FORMULAR UNA BUENA SOLICITUD A LA IA?

Un buen prompt técnico obtiene mejores respuestas.



CONTEXTO

¿De qué problema o situación estamos hablando?

1



OBJETIVO

¿Qué queremos obtener?

2



DATOS

¿Qué información tenemos disponible?

3



RESTRICCIONES O SUPUESTOS

¿Qué condiciones o supuestos debe respetar la solución?

4



TAREA

¿Qué queremos que haga exactamente la IA?

5



CRITERIOS DE VERIFICACIÓN

¿Cómo comprobaremos que la solución es correcta?

6



FORMATO DE RESPUESTA

¿Cómo queremos recibir la respuesta?

7

La calidad de la respuesta depende de la claridad de tu solicitud.



SOLICITUD POCO EFECTIVA

“Hazme un programa para calcular una viga.”



VS



SOLICITUD TÉCNICA EFECTIVA

“Para una viga simplemente apoyada de 6 m sometida a una carga uniformemente distribuida de 10 kN/m, propón un algoritmo para calcular las reacciones y los diagramas de cortante y momento. Explica primero el procedimiento, después implementa la solución en Python y verifica las reacciones mediante las ecuaciones de equilibrio. Presenta los resultados en kN y kN·m.”



- ✗ Sin contexto.
- ✗ Sin datos.
- ✗ Sin objetivo claro.
- ✗ Sin restricciones o supuestos.
- ✗ Sin tarea específica.
- ✗ Sin criterios de verificación.
- ✗ Respuesta genérica e incompleta.

- ✓ Contexto claro.
- ✓ Datos definidos.
- ✓ Objetivo específico.
- ✓ Restricciones o supuestos explícitos.
- ✓ Tarea específica y ordenada.
- ✓ Criterios de verificación.
- ✓ Formato de respuesta deseado.



Un buen prompt técnico no reemplaza el conocimiento del problema: exige expresarlo con claridad.



Piensa como **ingeniero**, pregunta como **ingeniero**, verifica como **ingeniero**.



LA IA ES UN ASISTENTE PODEROSO, pero el ingeniero dirige el proceso.



Una buena pregunta produce una buena respuesta.



Verifica siempre los resultados: no asumas, comprueba.

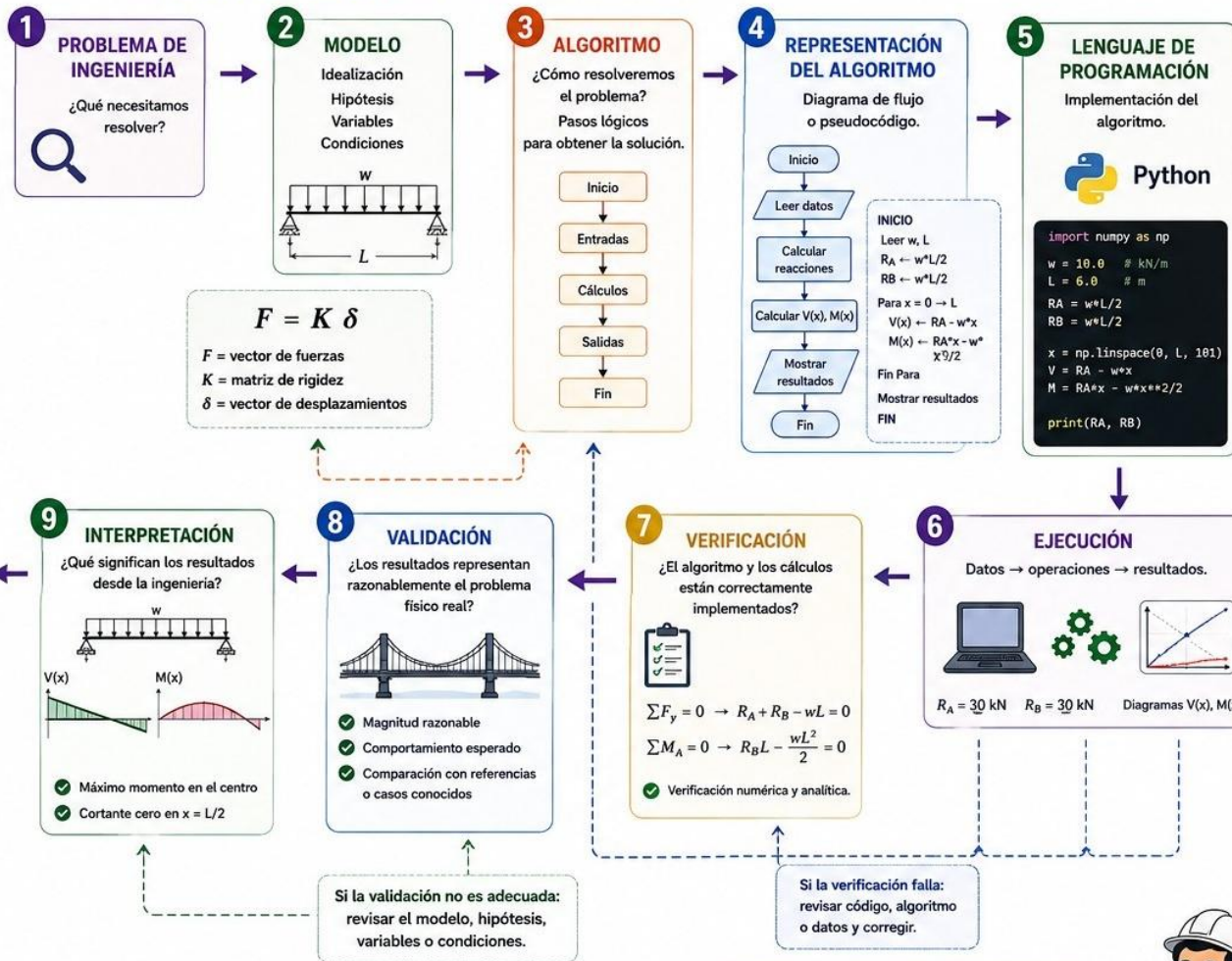


TU RESPONSABILIDAD FINAL: la seguridad y el buen juicio ingenieril.

De una idea al programa: el proceso completo



Estructura real
y su entorno



IA GENERATIVA
ASISTE DURANTE TODO
EL PROCESO



Comprender el problema
y explicar conceptos.

Proponer algoritmos
y pseudocódigo.

Generar y documentar
código en Python.

Depurar y corregir
errores.

Verificar cálculos
y procedimientos.

Visualizar resultados
y crear gráficos.

Interpretar resultados
y redactar reportes.

La IA puede asistir,
pero NO sustituye
el criterio ni la
responsabilidad del
ingeniero.



PROGRAMAR EN INGENIERÍA ES TRANSFORMAR UN PROBLEMA REAL
EN UNA SOLUCIÓN COMPUTACIONAL VERIFICABLE, INTERPRETABLE Y ÚTIL.

Flujo principal del proceso
Retroalimentación por verificación
Retroalimentación por validación



El resultado final
es tan bueno como
el modelo, el método
y los datos que
utilizamos.

De los primeros programas estructurales a la automatización

Una línea de tiempo de la evolución del análisis estructural asistido por computador

ANTES DE LOS 60 CÁLCULOS MANUALES Métodos clásicos



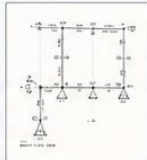
- Tablas y reglas de cálculo
- Métodos de rigidez y flexibilidad manuales
- Diagramas dibujados a mano
- Alto consumo de tiempo
- Propensos a errores humanos

Base indispensable:
comprender el método
y la física del problema.

AÑOS 60 | INICIOS DE LA AUTOMATIZACIÓN

STRESS

1963 (desarrollado) – 1964/65 (publicado MIT)
Structural Engineering Systems Solver
(MIT – ICES*)



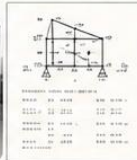
- Uno de los primeros programas de análisis estructural para uso general.
- Enfoque orientado al "problema" (Problem Oriented System).
- Permite al ingeniero describir la estructura en términos de ingeniería.
- Análisis de estructuras planas.
- Pionero del uso de matrices en el cálculo estructural.

Hito: llevó los métodos matriciales
del papel al computador.

AÑOS 60 | EXPANSIÓN Y LENGUAJE ESTRUCTURAL

STRUDL

1966 (primeras referencias) – 1968 (manuales MIT)
Structural Design Language
(MIT – ICES*)



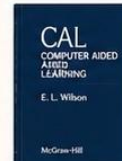
- Lenguaje más general para describir y analizar estructuras.
- Mayor capacidad para modelos más complejos (pórticos, rejillas, vigas continuas, etc.).
- Extensión natural de la filosofía iniciada con STRESS.
- Muy influyente en la investigación y en la práctica.

Hito: un lenguaje estructural que permitió
modelar y resolver problemas más generales.

FINALES DE LOS 70 | HITO PEDAGÓGICO

CAL

1979 (publicación)
Computer Aided Learning
(Edward L. Wilson)



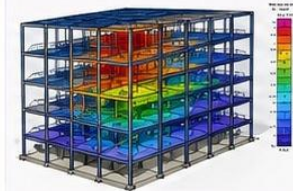
- Concebido para cerrar la brecha entre la enseñanza de los métodos matriciales tradicionales y el análisis automatizado.
- Enfocado en la comprensión del método a través de ejemplos guiados.
- Herramienta educativa que formó a generaciones de ingenieros.

Hito pedagógico: el computador como
apoyo para aprender y entender el método.

DESDE LOS 90 HASTA HOY | AUTOMATIZACIÓN TOTAL

SOFTWARE MODERNO

ETABS – SAP2000 – COMBAT
y otros



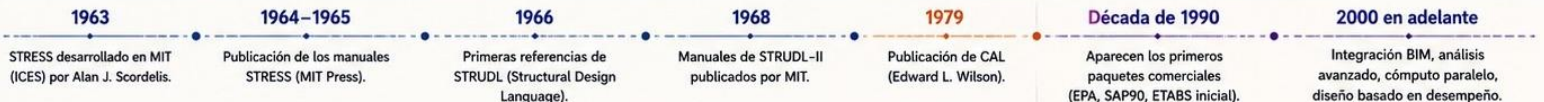
- Modelos tridimensionales completos.
- Métodos avanzados: MEF, P-Δ, no linealidad, dinámica, espectral, tiempo-historia.
- Integración gráfica avanzada y modelado BIM.
- Análisis de múltiples casos y combinaciones.
- Resultados rápidos, visuales y confiables.

Hito actual: análisis complejos en segundos
con gran potencia y versatilidad.

¿POR QUÉ FUE IMPORTANTE ESTA EVOLUCIÓN?

- Reducción drástica del tiempo de cálculo
- Mayor precisión y confiabilidad
- Modelos más realistas y complejos
- Mejor evaluación de seguridad y desempeño
- Apoyo fundamental a las decisiones de ingeniería

CRONOLOGÍA CLAVE



De las reglas de cálculo al análisis tridimensional avanzado: una misma idea guía esta historia –
convertir el conocimiento en herramientas para entender y diseñar mejor.

* ICES: Institute for Computer Applications in Engineering and Science (MIT), pionero mundial en cómputo aplicado a la ingeniería.



Software moderno de análisis estructural

Ejemplos de la evolución hacia análisis tridimensional, dinámico y por MEF

El computador permite modelar, analizar y comprender estructuras cada vez más complejas.



ETABS (Desde 1984)

Edificios y estructuras de gran altura



Modelado 3D integral de edificaciones.

Análisis estático y dinámico.

Diseño por código (AISC, ACI, NSR-10, Eurocódigos, etc.).

Interacción suelo-estructura.

Resultados avanzados y detallados.

Aplicaciones típicas

- Edificaciones de concreto y acero
- Torres y rascacielos
- Placas, muros, losas y cimentaciones
- Análisis sísmico y de viento

SAP2000 (Desde 1976)

Puentes y estructuras generales



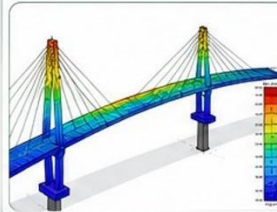
Análisis lineal y no lineal.

Pushover, espectral, tiempo-historia.

Combinaciones de carga y envolventes.

Modelación 3D flexible y parametrizada.

Verificación y diseño de múltiples materiales.

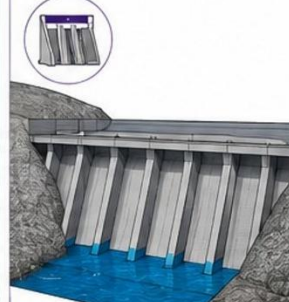


Aplicaciones típicas

- Puentes y pasarelas
- Estructuras industriales
- Cubiertas y domos
- Estructuras especiales
- Análisis sísmico y de estabilidad

COMBAT (Desarrollado en Colombia)

Originalmente para análisis de presas



Análisis por MEF para estructuras hidráulicas.

Análisis dinámico y sísmico.

Interacción estructura-fundación.

Consideración de presiones de agua y filtraciones.

Evaluación de seguridad y estabilidad.

Aplicaciones típicas

- Presas de concreto
- Estructuras hidráulicas
- Embalses y vertederos
- Análisis estático, sísmico y estabilidad
- Ingeniería del agua

¿QUÉ APOORTE TRAEN ESTAS HERRAMIENTAS?



Mayor capacidad de modelado y detalle.



Análisis más realistas y confiables.



Evaluación de comportamientos complejos.



Optimización de diseños y seguridad.



Resultados claros para la toma de decisiones.



El software es una herramienta poderosa, pero el ingeniero interpreta, juzga y toma decisiones.

EVOLUCIÓN CLAVE HACIA EL PRESENTE



Cálculo manual



Programas 2D básicos (STRESS, STRUDL)



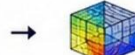
Programas matriciales (CAL)



Modelado 3D y análisis lineal



Análisis dinámico y no lineal



Métodos avanzados (MEF, elementos finitos)



Integración BIM y trabajo colaborativo



IA y aprendizaje automático

TENDENCIA ACTUAL



Modelos integrados arquitectura-estructura (Multi-disciplinar).



Análisis tiempo-historia no lineal y desempeño sísmico.



Computación paralela y nube: más potencia, menos tiempo.



BIM + Análisis + Diseño integrado para todo el ciclo de vida.



IA para optimización, detección de patrones y toma de decisiones.



EL SOFTWARE NO REEMPLAZA EL CRITERIO DEL INGENIERO; LO POTENCIA.



Nota histórica:

STRESS (1963) y STRUDL (1966) – pioneros en el uso del computador en análisis estructural (MIT – ICES).

CAL (1979) – hito pedagógico que cerró la brecha entre la enseñanza de métodos matriciales y la automatización.

Desde los 90 – software moderno con modelos 3D, dinámicos, no lineales, y hoy integrados con BIM e IA.



¿QUÉ HAY DETRÁS DEL SOFTWARE?

Del modelo físico al resultado: matemáticas, algoritmos y datos



El software es una interfaz amigable, pero el corazón es ingeniería, matemáticas y programación.



1 REALIDAD FÍSICA

Estructura real



- Geometría
- Materiales
- Cargas y acciones
- Condiciones de apoyo
- Comportamiento estructural

2 MODELADO

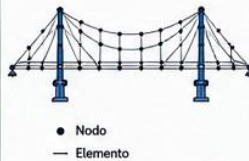
Idealización



- Líneas, áreas, sólidos
- Propiedades de secciones
- Asignación de materiales
- Apoyos y restricciones
- Hipótesis de análisis

3 DISCRETIZACIÓN

Modelo matemático (MEF)



- División en elementos finitos
- Grados de libertad (GDL)
- Funciones de forma
- Matrices elementales

4 FORMULACIÓN

Ecuaciones de equilibrio

$$K \delta = F$$

Matriz de rigidez global ($n \times n$) Vector de desplazamientos ($n \times 1$) Vector de fuerzas ($n \times 1$)

Obtención de K y F

- Integración numérica
- Transformaciones de coordenadas
- Ensamblaje de matrices
- Aplicación de condiciones de borde

5 RESOLUCIÓN

Algoritmos numéricos

Resolver el sistema
 $K \delta = F$



- Eliminación gaussiana
- Descomposición LU
- Cholesky
- Métodos iterativos

Obtención de desplazamientos δ

6 POSTPROCESO

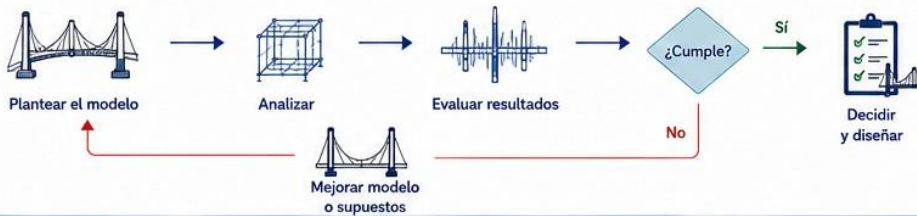
Resultados



Tablas, gráficos, reportes, modelos 3D interactivos

EL CICLO DE ANÁLISIS ESTRUCTURAL

Es un proceso iterativo



RECUERDA:

El software ejecuta el proceso, pero el ingeniero:

- Define el problema
- Construye el modelo
- Interpreta los resultados
- Toma decisiones

NIVELES DE SOFTWARE ESTRUCTURAL

BÁSICO (2D estático)

Marcos, pórticos planos, cargas simples.



INTERMEDIO (2D y 3D estático)

Análisis lineal elástico, combinaciones de carga, diseño.



AVANZADO (3D dinámico)

Comportamiento dinámico, espectral, tiempo-historia, no lineal.



ESPECIALIZADO (MEF avanzado)

No linealidad de materiales y geometría, contacto, fractura, etc.



INTEGRADO (BIM) Y COLABORATIVO

Información del ciclo de vida, coordinación y gestión.



EMERGENTE (IA y automatización)

Optimización, detección de patrones y apoyo a decisiones.



¿QUÉ LO HACE POSIBLE?



Matemáticas
Álgebra lineal, cálculo numérico, teoría de matrices.



Algoritmos
Métodos eficientes para resolver sistemas grandes.



Programación
Códigos optimizados en C/C++, Python, Fortran, etc.



Estructuras de datos
Almacenamiento compacto y eficiente de grandes matrices.



Interfaz gráfica
Visualización 3D, entrada de datos, generación de reportes.



Verificación y validación
Asegura precisión, confiabilidad y seguridad.

SIN FUNDAMENTO TÉCNICO, NO HAY BUEN ANÁLISIS.



Un modelo mal planteado o mal interpretado puede llevar a resultados engañosos y decisiones inseguras.



EL SOFTWARE ES UNA HERRAMIENTA PODEROSA, PERO EL CONOCIMIENTO, EL JUICIO Y LA ÉTICA DEL INGENIERO SON INSUSTITUIBLES.



Comprender lo que hay detrás del software te convierte en un mejor modelador, analista e ingeniero.